

Assignment 2: Process Numerical Data

Fri Sep 4 2026

This lab treats NumPy as part of a small data-engineering pipeline rather than as a collection of disconnected array exercises. The underlying pattern is

raw data → **validation** → **filtering** → **transformation** → **aggregation** → **quality output**

This pattern provides a natural bridge to later topics such as Pandas, schemas, ETL/ELT, data-quality rules, vectorized processing, and distributed processing. At the end of the lab, you will consider why the same operations become more complicated when a sensor batch contains **10 billion observations rather than 10**. This question is the transition from in-memory processing to scalable data engineering.

Learning Objectives

- Inspect the shape, dimensionality, size, and data type of a NumPy array;
- Validate and filter numerical records with Boolean masks;
- Transform columns with vectorized operations and broadcasting;
- Aggregate measurements along the correct axis; and
- Produce a concise data-quality summary for a processed batch.

Scenario

You are a data engineer supporting a manufacturing facility. Every minute, machines send a batch of sensor readings to a processing system. Each row is one machine observation, and each column has the following meaning:

Column	Field	Unit
0	Machine ID	identifier
1	Temperature	°C
2	Vibration	mm/s
3	Pressure	kPa
4	Energy usage	kWh

The raw batch is stored in a NumPy array. The sentinel value **-999** represents a missing sensor reading.

```
import numpy as np

data = np.array([
    [101, 72.4, 2.1, 101.3, 12.5],
    [102, 85.1, 5.8, 98.7, 15.2],
    [103, 69.8, 1.9, 102.1, 11.8],
    [104, 91.3, 7.2, 97.4, 18.1],
    [105, 76.5, 3.0, 100.5, 13.7],
    [106, -999, 4.1, 99.9, 14.8],
    [107, 88.7, 6.5, 96.8, 17.3],
    [108, 73.2, 2.4, 101.8, 12.9],
    [109, 79.6, 3.8, 100.1, 14.2],
    [110, 95.2, 8.1, 95.9, 20.4]
])
```

Plan

	Work
Understand	Read the schema and inspect the raw batch
Process	Validate, filter, and transform the data
Analyze	Aggregate measurements and flag machines
Report	Produce the quality summary and discuss scale

Engineering Constraint

For Tasks 3–9, do not use explicit Python `for` loops. Use NumPy indexing, Boolean masking, vectorized expressions, broadcasting, and aggregations. You may print intermediate values while developing your solution.

Tasks

- Task 1. Inspect the batch.** Report `data.shape`, `data.ndim`, `data.size`, and `data.dtype`. Explain why Machine ID is stored as a floating-point value even though its values look like integers.
- Task 2. Separate identifiers from measurements.** Create a one-dimensional array named `machine_ids` containing the first column. Create a two-dimensional array named `measurements` containing temperature, vibration, pressure, and energy usage. Confirm the shape of each result.
- Task 3. Validate the records.** Create a Boolean mask that identifies every row containing `-999` in its measurement columns. Report the Machine ID associated with each invalid row.
- Task 4. Filter the batch.** Use the validation mask to create `clean_ids` and `clean_measurements`. Both arrays must contain only valid records and must remain aligned row by row.
- Task 5. Aggregate the clean data.** For each measurement column, calculate its mean, minimum, and maximum. The output must summarize temperature, vibration, pressure, and energy usage separately. Pay close attention to the `axis` argument.
- Task 6. Transform temperature.** Convert the clean temperature values from Celsius to Fahrenheit using

$$F = \frac{9}{5}C + 32.$$

Store the result in a new array. Do not overwrite the Celsius measurements.

- Task 7. Apply a quality rule.** A machine requires inspection when either

$$\text{temperature} > 85^{\circ}\text{C} \quad \text{or} \quad \text{vibration} > 6.0 \text{ mm/s}.$$

Combine Boolean conditions to create an inspection mask, then report the corresponding Machine IDs.

- Task 8. Center the measurements.** Calculate the mean of each clean measurement column and subtract those means from the clean data using broadcasting:

$$X_{\text{centered}} = X - \bar{X}.$$

Verify with NumPy that every column mean in the centered array is approximately zero. Briefly explain why tiny nonzero values may appear.

Task 9. Produce a quality summary. Print a final report containing:

- number of raw records;
- number of clean records;
- number of rejected records; and
- number of machines flagged for inspection.

Add one programmatic consistency check showing that clean and rejected record counts sum to the raw record count.

Deliverable

Submit one notebook file containing your code, printed results, and short written responses. Organize the work so a reviewer can follow this flow:

raw batch → validation → filtering → transformation → aggregation → quality summary

Early-Finishers

Assign each clean machine the health score

$$\text{score} = 0.4(\text{temperature}) + 3(\text{vibration}) + 0.5(\text{energy}).$$

Calculate every score with a vectorized expression. Use `np.argmax` to identify the Machine ID with the highest score. Then answer: Would this score still be meaningful if temperature, vibration, and energy were measured on dramatically different numerical scales? Why or why not?

Exit Discussion: From 10 Rows to 10 Billion

Suppose the pipeline receives 10 billion observations instead of 10. In two or three sentences, identify at least two complications. Consider memory capacity, data partitioning, parallel execution, data movement, failure recovery, and the cost of rescanning the full dataset. Which parts of this NumPy pipeline would remain conceptually the same, even if the implementation changed?

Points

Ten points: one point for each task, one point for the final discussion.